

Chapter III

TECHNICAL BACKGROUND

Backend Technologies Used

Visual Studio Code (VS Code). This is the code editor the researchers used to write, debug, and manage all of the backend's Java source code, SQL scripts, and configuration files.

Java. This is the programming language used to write the entire backend, including the logic that checks scanned answers, computes scores, and builds analytics reports.

Spring Boot. This is the Java framework used to build the backend as a set of RESTful API services that the mobile app and the web dashboard connect to.

Spring Web (Spring MVC). This is the part of Spring Boot used to expose the backend's features as REST API endpoints, such as downloading an assigned class to the mobile app or generating a report for the web dashboard.

Spring Security. This is the library used to check a logged-in user's role (teacher or principal) before allowing a request through, and to protect stored teacher passwords.

Spring JDBC (JdbcTemplate). This is the part of Spring used to connect the backend directly to the database and run the SQL statements that retrieve and save records such as students, assessments, and scanned answers.

Jakarta Bean Validation (Spring Boot Validation). This is the library used to check that data sent from the mobile app or the web dashboard is complete and

correctly formatted before the backend processes it.

Spring Boot Mail. This is the library used to send account verification and registration emails to newly created teacher accounts.

Spring Boot DevTools. This is the tool used to automatically restart the backend server whenever the researchers saved a code change during development.

MySQL / MariaDB. This is the relational database management system used to permanently store all of the system's data, such as users, classes, students, assessments, scanned answers, and analytics results.

Apache XAMPP. This is the local server package used to run MySQL/MariaDB and phpMyAdmin on the researchers' own computers for developing and testing the backend before deployment.

H2 Database. This is the in-memory database used only while running the backend's automated tests, so testing never affects the real school data.

Apache PDFBox. This is the Java library used to generate the printable bubble answer sheets and the downloadable PDF reports, such as the class record and item analysis.

Apache POI. This is the Java library used to generate the downloadable Excel (.xlsx) versions of the system's reports.

Google ZXing (Zebra Crossing). This is the QR code library used to generate the unique QR code printed on every answer sheet, which the mobile app scans to identify the exact test and student it belongs to, as well as the QR code used to set up

two-factor authentication.

Jackson. This is the library, bundled with Spring Web, used to convert the backend's data to and from JSON whenever it exchanges information with the mobile app or the web dashboard.

Brevo. This is the third-party email delivery service connected to Spring Boot Mail so that verification and registration emails actually reach a teacher's real inbox.

Maven. This is the build tool used to manage the backend's external libraries and to compile, package, and run the application.

Postman. This is the tool used to manually test the backend's REST API endpoints during development, before the mobile app and web dashboard were connected to them.

Git and GitHub. These are the version control tool and hosting platform used to track source code history and to share and combine backend, mobile, and web dashboard code among the research team.

Mobile Application Technologies Used

React Native. This is the cross-platform mobile application framework the researchers used to build the entire teacher-facing mobile app, compiling the JavaScript/TypeScript source into a native Android application for scanning, verifying, and syncing test results.

TypeScript (TS). This is the typed superset of JavaScript the researchers used to write the mobile app's source code, catching type mismatches, such as in the data sent to the backend during sync, at compile time before they could cause a failure on a

teacher's device.

React. This is the underlying UI library React Native is built on, used throughout the app to manage each screen's state and re-render it automatically whenever the teacher's data changes, such as the student list, scan-review screen, and synchronization screen.

Kotlin. This is the language the researchers used to write the mobile app's custom native Android modules, namely the OMR bubble/QR-code detector, the camera-capture bridge, and the secure session-storage module, for functionality that JavaScript alone cannot perform on the device.

react-native-quick-sqlite. This is the local SQLite database library the mobile app uses to store downloaded class, test, and scan data directly on the teacher's phone, so a teacher can scan and check student answers without an active internet connection and upload the results later.

OpenCV (Open Source Computer Vision Library). This is the image-processing library, used from the app's native Kotlin code, that detects the printed page markers on a scanned answer sheet, corrects the photo's perspective and lighting, and determines whether each answer bubble is shaded in.

ZXing (Zebra Crossing). This is the barcode/QR-code library the app uses, alongside OpenCV, to decode the QR code printed on each answer sheet, which identifies the exact student, test, and page being scanned.

Hermes. This is the JavaScript engine React Native uses to run the app's compiled TypeScript/JavaScript code on the Android device, chosen for its faster startup time on mobile hardware.

Android Keystore. This is the Android operating system's secure hardware-backed storage, used through a custom native module to keep a signed-in teacher's session token encrypted on the device instead of in plain, readable storage.

lucide-react-native. This is the icon library the researchers used to render the app's interface icons, such as the camera, checkmarks, search, and warning banners, consistently across every screen.

react-native-svg. This is the vector-graphics library that renders lucide-react-native's icons and any other scalable graphics the app needs.

react-native-safe-area-context. This is the library the researchers used to keep the app's buttons and text clear of a phone's notch, status bar, and rounded screen corners.

sha.js. This is the hashing library the app uses to compute SHA-256 hashes of downloaded class and test data, producing a stable identifier for each offline snapshot and detecting whether a payload changed since it was last downloaded.

Metro. This is React Native's JavaScript bundler, which compiles and serves the app's TypeScript/JavaScript source code to the physical Android device during development.

Jest. This is the testing framework the researchers used to write and run the mobile app's automated tests, covering its sync logic, local database repositories, and backend-contract parsing.

ESLint. This is the static-analysis tool used to catch problematic JavaScript/TypeScript patterns in the mobile app's source code before they are

committed.

Prettier. This is the code-formatting tool the researchers used to keep the mobile app's source code consistently styled across every file.

Babel. This is the JavaScript compiler React Native uses, through its own preset, to convert the app's TypeScript/JavaScript syntax into a form the Android JavaScript engine can run.

Android Studio / Gradle. This is the native Android build toolchain the researchers used to compile the app's Kotlin native modules, manage its Android dependencies such as OpenCV and ZXing, and package the app for installation on a physical device.

Android Debug Bridge (ADB). This is the command-line tool the researchers used to install and run the app on a physical Android phone, forward the phone's network requests to the backend server running on the development computer, and inspect on-device logs while diagnosing issues.

Visual Studio Code (VS Code). This is the code editor the researchers used to create, view, and modify the mobile app's source code.

Git and GitHub. This is the version-control system the researchers used to track changes to the mobile app's source code and coordinate work between the mobile, backend, and web components of the system.

Web Dashboard Technologies Used

Visual Studio Code (VS Code). This is the code editor the researchers used to write, debug, and organize the web dashboard's React source code and configuration

files.

JavaScript (JS). This is the language the entire web dashboard is written in, covering every page, API call, and component, using the JSX syntax extension for building UI markup; the project uses plain JavaScript, not TypeScript.

React. This is the JavaScript library used to build the dashboard's interface as reusable components (tables, modals, forms, charts), rendered via React DOM and driven by React's own state and hooks (useState, useEffect, useContext) rather than a separate state-management library.

Vite. This is the build tool and development server used to bundle the web dashboard for production and to serve it with fast reloading while the researchers were developing it.

Tailwind CSS. This is the utility-first CSS framework used to style every page and component directly in JSX class names, wired into the project through its official Vite plugin.

shadcn/ui. This is the component design system used to generate the dashboard's base UI elements (buttons, inputs, selects, cards, badges), built on Radix UI primitives and customized with Tailwind CSS.

Radix UI. This is the library of unstyled, accessible component primitives (Label, Popover, Select, Slot) that the dashboard's shadcn/ui components are built on top of, providing keyboard navigation and accessibility behavior.

class-variance-authority (CVA). This is the utility used to define style variants, such as different button colors and sizes, for the dashboard's reusable UI

components.

clsx. This is the utility used to conditionally combine CSS class names in the dashboard's components, such as toggling active or disabled styles.

tailwind-merge. This is the utility used alongside clsx to resolve conflicting Tailwind classes so that component style overrides apply correctly throughout the dashboard.

Recharts. This is the charting library used to render the report visualizations, such as item analysis and performance charts, on the dashboard's Reports page.

Lucide React. This is the icon library used for interface icons throughout the dashboard's buttons, navigation, and status indicators.

React DatePicker. This is the component library used for date and time selection fields in the dashboard, such as scheduling assessments.

Fetch API. This is the built-in browser API used as the HTTP client for all communication between the web dashboard and the backend's REST endpoints, in place of a separate library such as Axios.

ESLint. This is the linting tool used to catch errors and enforce consistent coding rules, including React Hooks usage, in the dashboard's source code.

Git and GitHub. These are the version control system and repository host used to track changes to and manage the web dashboard's codebase.